

Lab 4

V2: What's new. Shots. Team name output & messages.

V3: Using the linked list. Re-ordered content

V4: Test files. Bonus.

V5: Sort removed. Output files are given in Piazza. Clarification of bonus 1

AWS related: New dates, linked list is bonus

Dates

Prototypes demo by Thu 24-Oct-25

Early Fri 24-Oct-25

On-Time: Wed 29-Oct-25

Late until Thu 30-Oct-25

Contents

Lab 4.....	1
V2: What's new. Shots. Team name output & messages.	1
V3: Using the linked list. Re-ordered content.....	1
V4: Test files. Bonus.	1
V5: Sort removed. Output files are given in Piazza. Clarification of bonus 1	1
AWS related: New dates, linked list is bonus.....	1
Dates	1
Prototypes demo by Thu 24-Oct-25	1
Early Fri 24-Oct-25	1
On-Time: Wed 29-Oct-25.....	1
Late until Thu 30-Oct-25	1
The New Stuff in Lab 4	4
1. Team names instead of color numbers.....	4
2. Command line parameters: the input file name, the output file name, and possibly another token to enable second bonus mode	4
3. Input and output files that you must open and close	4
4. Shots fired by the ships and kept using the supplied linked list library	4
5. A linked list that you write to replace the linked list library (optional bonus).....	4

6. Shots will hit rocks and break them into smaller rocks (optional bonus)	4
7. Unreliable dynamic memory allocation for shots and linked list nodes so that your error detection and recovery gets put to use	4
Output Changes for Lab 4 – See supplied files before writing new code	4
Strings	5
Formatting	5
Messages.....	5
Tabular output	6
Diagnostics & Debug	6
I/O & Command Line Arguments.....	6
The command line must have the right number of arguments.....	7
Dealing with those files.....	7
Input.....	7
The input file must open for reading	7
After reading, the input file must be closed	8
Output.....	8
The output file must open for writing	8
After writing, the input file must be closed	9
Reading user input – part of the second bonus.....	9
Shots.....	9
Creating a shot	9
Adding the shot to the linked list.....	10
Moving shots.....	10
Hitting rocks – PART OF BONUS #1	10
Fission – THE OTHER PART OF BONUS #1	11
Getting the separation values.....	11
Applying the separation values.....	12
Add the new rocks	12
Final Actions – still required.....	12
The Linked List.....	12
Head Pointer Operations	13
Data on the list.....	13

Callback functions	13
Action Functions	14
Comparison Function	14
Criteria Functions	14
Library functions	14
Insert	14
deleteSome	14
Iterate.....	15
Other Functions	15
Building with the list library	15
Prototypes and the list.....	15
Interactions Between Structs and the Lists	15
The sim structure and the list	16
Functions that change the list heads	16
Functions that don't change the list heads.....	16
Clarifications	16
Initializations	17
Implementing the Linked List (optional 10 point bonus).....	17
Signatures	17
Files	17
Dynamic Memory for the list	17
Visibility.....	18
Plan of Attack.....	18
Terse UPDATED :.....	18
Ten-line Limit	18
Unreliable Allocation.....	19
Dealing with allocation failures.....	19
Recovery.....	19
Details	19
Code Changes.....	19
Prototypes for altmem.....	20
Initial testing for allocation failures	20

Reference code issues.....	20
Bonus #1: Shots hitting rocks: 10 points.....	21
Bonus #2: Bonus Mode: A Playable Game: 10 points.....	21
Bonus #3, 10 points – implement the linked list, see above	22
Test Files.....	22
No Global Variables! Code Must Compile! No Magic Numbers!	23
Submission	24

The New Stuff in Lab 4

Lab 4 will have:

1. Team names instead of color numbers
2. Command line parameters: the input file name, the output file name, and possibly another token to enable second bonus mode
3. Input and output files that you must open and close
4. Shots fired by the ships and kept using the supplied linked list library
5. A linked list that you write to replace the linked list library (optional bonus)
6. Shots will hit rocks and break them into smaller rocks (optional bonus)
7. Unreliable dynamic memory allocation for shots and linked list nodes so that your error detection and recovery gets put to use

As an optional bonus, up to 3 people can fly the ships.

It might be best to implement these things in roughly the order they are listed. *Do not do more than one at a time. Controlling complexity is key. When you get one of them working, make a zip file suitable for turning in. When you get the next one working, rebuild your zip. It's fine to turn the lab in multiple times – we grade the last one you submit.* Team names is straightforward. You can't open and close the files without first processing the command line parameters. You don't have to do shots hitting rocks when you first do shots.

Output Changes for Lab 4 – See supplied files before writing new code

Much of the code for this is given in the supplied files for lab 4: `output.c`, `text.c` `graphics.c`. You are still on the hook for making sure that your code calls the supplied code. Be sure to read this section for items that would not be in those three files, such as diagnostics generated by your linked list code.

The biggest output change is that the color of the object will be printed as a string in most of the places where the program does output.

Strings

Your program will need to have a function that will return a string when given a color number. Consider an array of strings. Use the following chart to map color numbers to color names. Your strings will not be in all capital letters, just have the first letter capitalized.

```
/* curses basic colors:
 * BLACK  0
 * RED    1
 * GREEN  2
 * YELLOW 3
 * BLUE   4
 * MAGENTA 5
 * CYAN   6
 * WHITE  7
 */
```

Formatting

Use %7s to print the color name in tabular output for rocks, ships, and shots:

```
fprintf(fp,
"%7s Shot at (%7.4lf, %7.4lf), Velocity: (%6.2lf, %6.2lf)\n",
    team(shot->color), shot->pv.x_position, shot->pv.y_position,
    shot->pv.x_velocity, shot->pv.y_velocity);
```

The above output uses a function called “team” that you would have to write. You don’t have to match the reference output in terms of whitespace but it is nicer for the graders. Example output below. See output files for complete examples.

Using the input file xZB.rx after 1 update we get:

```
Time:  0.0625
Magenta Rock at (22.3180, 10.6820), Velocity: ( 0.00, 0.00), Mass: 4.0
Cyan Rock at (13.0000, 10.0000), Velocity: ( 0.00, 0.00), Mass: 5.0
Blue Rock at (28.6820, 4.3180), Velocity: ( 0.00, 0.00), Mass: 5.0

Red Ship at (27.9375, 15.0000), Velocity: ( -1.00, 0.00), Angle: 225 Score: 0
Green Ship at (28.9375, 15.0000), Velocity: ( -1.00, 0.00), Angle: 300 Score: 0
Yellow Ship at (26.9375, 15.0000), Velocity: ( -1.00, 0.00), Angle: 275 Score: 0

Yellow Shot at (26.5019, 15.0436), Velocity: ( -7.97, 0.70)
Red Shot at (27.6464, 14.6464), Velocity: ( -5.66, -5.66)
Green Shot at (28.5670, 15.2500), Velocity: ( -6.93, 4.00)
```

Messages

Retain all existing messages and **update** the text mode and graphics mode messages. Use %s (not %7s) to format the color names. Your code **must** support the first 4 messages below. The first bonus also requires the hit message.

```
FIRE! Red ship at (28.0000, 15.0000) fires at angle 225.0
BUMP! Magenta rock at (23.0000, 10.0000) hits Blue rock (28.0000, 5.0000)
CRUNCH! Green ship at (10.1875, 15.0000) hits Cyan rock at (11.2496, 15.6352)
Green shot at leaves play at (20.3397, 20.0000)

HIT! Red shot at (25.53, 12.53) hits Magenta rock size 4.00 at (22.3180, 10.6820)
```

For the graphics versions of these messages, use fewer decimal places (two or less) to shorten the messages so that they fit.

Tabular output

Add color name to the regular and final tables as described above.

Diagnostics & Debug

See the linked list, command line arguments, and unreliable sections for information and examples. The list code gets diagnostics similar to the dynamic memory code for objects. **The list code also gets mandates for debug messages.**

The value of seeing these pointers is if free blows up you can tell what kind of values should have been passed to it. Add similar messages to the linked list code:

```
DEBUG: linkedlist: allocated pointer is 0xf64190
DEBUG: linkedlist: about to free 0xf642d0
```

You might want to add the debug messages in “add_shot.” That are not mandated but they do give good visibility into what is going on.

Here are some of the diagnostics needed by file IO:

```
DIAGNOSTIC: Successfully opened xZ1.rx for reading.
DIAGNOSTIC: Successfully opened xZ1.bonus.text for writing.
DIAGNOSTIC: Input file closed
DIAGNOSTIC: Output file closed
```

There are also required ERROR messages related to file operations.

I/O & Command Line Arguments

To clarify: DIAGNOSTIC and ERROR messages go to stderr and your code doesn't have to check anything to be able to print them. True DEBUG messages have to pay attention to the DEBUG symbol, and they also go to stderr.

In lab 4, all objects will be read from a file whose name is given on the command line. Some input might be read directly off the keyboard for a bonus. All normal text output goes to a file that is also given on the command line. Once that is the case, your lab can have both TEXT and GRAPHICS be true at the same time.

UPDATE: Your lab should be able to be run with both TEXT and GRAPHICS set to true at the same time. Make changes to mode_init in lab4.c:

- If GRAPHICS is true, return rx_init
- if GRAPHICS is not true, return TEXT

To run the lab, at a minimum you would use something like the following command:

```
[kirby.249@coe-dnc268475s lab4]$ ./lab4 xM.rx xM.text
```

For bonus mode, add another command line parameter. It can be anything, we historically use “bonus” because the presence of the additional parameter requests bonus mode. Since graphics is required to be enabled for bonus mode, we take control of stderr (file descriptor 2) and get it off screen and redirected to a file. (Redirect stderr anytime GRAPHICS is true)

```
[kirby.249@coe-dnc268475s lab4]$ ./lab4 xZB.rx xZB.text bonus 2>xZB.dnd
```

The command line must have the right number of arguments

The initialization checks in lab 4 are more extensive than lab 3.

- If argc is less than 3, the lab should give an error and decline to run.
- If argc is 4 and bonus mode is not present in the code, the lab should give an error and decline to run.
- If argc is 4 and the lab is in GRAPHICS is not true, the lab should give an error and decline to run regardless of whether bonus code is on-board or not.
- If argc is 4, the lab is in GRAPHICS mode, and the bonus code is on-board, the lab must run in bonus mode if all other initializations pass.

The output below is for when argc is less than 3 (the first bullet point above):

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4
ERROR: insufficient args. Usage: ./lab4 inputfile outputfile
Returning 1
Total runtime is 0.000086069 seconds
[kirby.249@coe-dnc268474s lab4]$
```

The output below is the same for the second and third cases given in the bullet points above:

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4 xZB.rx xZB.text bonus
ERROR: Bonus code is not present or graphics not enabled.
Returning 1
Total runtime is 0.000107050 seconds
[kirby.249@coe-dnc268474s lab4]$
```

Dealing with those files

Before calling master input, the sim should call a function to open the files. If it returns false, do everything will return false to main. The open files functions will call minions that open the input and output files and set the file pointers in the sim structure.

Your code **must** close any file that it successfully opens.

Input

The input file must open for reading

The input filename is given as argv[1]. If the file cannot be opened for reading, the lab should decline to run the simulation. If the file opens, a DIAGNOSTIC will be printed:

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4 xZB.rx xZB.text
DIAGNOSTIC: Successfully opened xZB.rx for reading.
```

If a file that cannot be opened is given on the command line, an ERROR message must be printed. Not being able to open the file means returning false up the calling chain. In the following output, there is no file named foo in the directory.

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4 foo bar
ERROR: Unable to open foo for reading.
Returning 2
Total runtime is 0.000636101 seconds
[kirby.249@coe-dnc268474s lab4]$
```

Use the **fopen** function to open the file. Google that function and read up on file I/O. Open it in read-only mode using “r” as the mode.

After reading, the input file must be closed

Just before the lab ends, close the input file pointer using **fclose** and print a DIAGNOSTIC message (if in TEXT mode). If your code opens the input file and then the output file, be sure to close the input file if the output file fails to open.

DIAGNOSTIC: Input file closed.

If the fclose call fails, print an error message.

ERROR: failed to close Input file.

Output

The output file must open for writing

The input filename is given as argv[2]. If the file cannot be opened for writing, the lab should decline to run the simulation. If the file opens, a DIAGNOSTIC will be printed:

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4 xZB.rx xZB.text
DIAGNOSTIC: Successfully opened xZB.text for writing.
```

If a file that cannot be opened is given on the command line, an ERROR message must be printed. Not being able to open the file means returning false up the calling chain. In the following output, there is no file named foo in the directory. In the following output, the file named bar is write protected. (Note that the code closed the input file when the output file failed to open).

```
[kirby.249@coe-dnc268474s lab4]$ ./lab4 xZ1.rx bar
DIAGNOSTIC: Successfully opened xZ1.rx for reading.
ERROR: Unable to open bar for writing.
DIAGNOSTIC: Input file closed
Returning 2
Total runtime is 0.001706362 seconds
[kirby.249@coe-dnc268474s lab4]$
```

Use the **fopen** function to open the file. Google that function and read up on file I/O. Open it in write mode using “w” as the mode.

After writing, the input file must be closed

After all text output completes, but certainly before the lab ends, close the output file pointer using **fclose** and print a DIAGNOSTIC message.

DIAGNOSTIC: Output file closed.

If the fclose call fails, print an error message.

ERROR: failed to close Output file.

Reading user input – part of the second bonus

If the bonus flag is set in the sim structure, your code will loop performing non-sleeping reads one character at a time using **rx_getch()** which is similar to **getchar()**. This call always returns immediately regardless of whether a typed-in character awaits on the input stream or not. If a character was waiting, that character will be returned. If no character was waiting, it returns ERR, a macro defined in <curses.h> as having a value of -1. It may be best to add the following line to input.c instead of including curses.h.

```
#define ERR      (-1)
```

If a value other than ERR is returned, process that character and then read again. If ERR is returned, ignore it and the loop is done. ERR means that the user hasn't pressed any keys since we read that last one. We will read all waiting characters so that we can do all of the teams at the same time.

The input.c file will need to include rx.h to get access to **rx_getch**.

Shots

Shots need a PandV structure, their team color number, and a pointer back to the sim structure.

Shots will be stored on the linked list. This means that the sim structure gains a void pointer that is the list head. [See list writeup] One of the few things the application code is allowed to do with the list head is to check it for NULL, which means that the list is empty.

In the core sim loop, right after updating the elapsed time and printing the corresponding debug message, it is time for ship actions. If the sim is not in bonus mode, ship actions will be to check the list head to see if it is NULL. If so, there are no shots in the game, so every ship needs to fire a shot. Use the existing ship iterator and have each ship fire a shot.

Creating a shot

When firing a shot, generate a message and then create the shot and then add the shot.

For the new shot, we will use dynamic memory. If allocation fails, we give up on that particular shot. The allocation code will detect the failure and print an error message.

If we successfully allocate the dynamic memory, we use ship data to compute the shot data. The shot is located at the same position as the ship. The shot color is the same as the ship color. Be sure to fill in

the sim pointer on the shot. The shot velocity is based on the angle of the ship and not at all on the ship velocity. Our shots have an overall velocity of 8, split into an X and Y component.

Shot x velocity = $8 * \sin(\text{ship angle converted to radians})$

Shot y velocity = $8 * \cos(\text{ship angle converted to radians})$

8 here is a **magic** number. The math.h header file has declarations for sin and cos. The math library is already present in our makefiles. To get radians, take degrees times PI and divide by 180.0. The following code needs to be in constants.h (if it isn't already) since we don't get access to M_PI in math.h due to the flags we use to compile.

```
#ifndef M_PI
#define M_PI (3.14159265358979323846)
#endif
```

Adding the shot to the linked list

Your code will use the insert function of the linked list to add the shot to the list. If insert fails, free the shot and go on. Your dynamic memory code will print a diagnostic when it does so. The shots are placed on the list in X position order, so you will need a comparison function to give the list that compares X positions.

Moving shots

After ship actions in the core sim loop, the core sim loop will do motion-related activities. The first of those is to do the basic movement of everything, including shots. Use the linked list iterate function and give it a action function that knows how to make basic motion happen on a shot. Unlike ships and rocks, shots do not wrap when they go out of bounds.

The second motion-related activity is to purge shots that are out of play. Use the list deleteSome functionality to do so. The criteria function will decide if the shot is out of play. The action function passed in will generate a leaves play message and then calls your dynamic memory code that frees the shot. (That dynamic memory code will have a diagnostic to print).

As before, the next motion-related activities are to smash ships and then bump rocks into rocks.

The final motion related activity is to have the shots see if they hit any rocks. Your code will call the linked list iterate function again to go through the shots. The action function will be for one shot hitting rocks and it will be in rocks.c since it's mostly about the rocks.

Hitting rocks – PART OF BONUS #1

The action function mentioned above will take a passed in shot pointer and go through the rocks in reverse order. We can't use the existing rock iterator this time. The loop will start with the last rock in the array (highest subscript) and work towards the beginning of the array (subscript zero).

For each such rock, the shot hit if the distance, center to center, is less than or equal to the rock mass which we use as its radius. If the shot hit the rock, we output a hit message and then points are scored. In the ship array in the sim structure, increment the score for the ship that has the same color as the shot. Cast the rock mass as an int and add that many points. A rock with mass 2.5 adds 2 points.

The next step after scoring points is to fission the rock. Fission is complex enough to be a separate function. It might generate two new rocks from our old rock (and it might not). After fission, we get rid of the original rock by copying the rock that is the last rock in the array over top of the rock that is going away. We also have to decrement the rock count.

Fission – THE OTHER PART OF BONUS #1

If the rock mass is equal or greater than 2.0, the rock will split into two rocks. Fission does nothing with smaller rocks. Your code will need to create two local rocks. Think of them as the left rock and the right rock. Copy the original rock data to the left rock and divide the mass by two. Copy the left rock to the right rock. Now we have two identical rocks that are at the same location. We need to separate them in space and have them move away from each other.

Our rock will split along a split line that goes from the contact point (where the shot is) radially to the center of the rock. This will ignore the shot's direction of travel and has no basis in real physics, but the math is far, far simpler and it usually looks pretty good onscreen. If the rock's direction of travel is toward the center of the rock, it looks plausible. The parts fly away from each other in opposite directions.



In the picture above the black o is a shot that hit the blue rock. The split line goes from the shot through the center of the rock. The after picture shows two rocks, each with half the radius, that touch where the center of the original rock used to be. The line segment connecting the centers of the new rocks is perpendicular to the split line. It has a length equal to the radius of the old rock. The new rocks will have their positions moved away from the original rock position along this line. The new rocks will travel away from each other along this line.

Getting the separation values

These new rocks are not centered where the old rock used to be. We can compute the offsets in X and Y. Given to you in a new physics.c file is the split_offset function.

```
double split_offset(double p1, double p2, double dist, double radius);
```

The p1 position will come from the original rock. The p2 position will come from the shot. The distance will be the center-to-center distance between the original rock and the shot. The radius is the radius of the original rock.

When we feed X values to this function, we will get the delta Y value as a result. In the above picture, we need to move the new rocks apart more in Y than in X because the separation line is perpendicular to the direction we want the rocks to move apart in. The difference in X between the shot position and the rock position is bigger than the difference in Y. The shot is closer to horizontal to the rock so the

separation of the halves will be mostly vertical. (The new rocks are trying to go away from the impact.) We can get the delta X number by feeding Y position data to the split_offset function.

Applying the separation values

It's probably best to have a separate function that does this since it involves 8 assignments.

Now that we have the delta X and delta Y values, we can adjust the positions of the left and right rock.

We will also add velocity components to the new rocks that will cause them to move away from each other along the line connecting their centers. In a total hand wave on units, we will use the delta X and delta Y we computed for the positions to get the values we add to the velocities without any scaling.

Here is how we apply those separation values:

The delta X value is • Subtracted from the left rock X position • Subtracted from the left rock X velocity • Add to the right rock X position • Added to the right rock X velocity	The delta Y value is • Added to the left rock Y position • Added to the left rock Y velocity • Subtracted from the right rock Y position • Subtracted from the right rock Y velocity
--	--

(If you want to experiment with different physics about how fast the rock fly apart, multiply some scaling factor on the changes to velocity, but keep the position changes since they are exact. Be sure that the code that you turn in has no such scaling.)

Add the new rocks

The fission function ends by adding the left rock and the right rock to the sim. You already have add rock code that input uses. To summarize, the fission function checks rock size and if it like it, it creates two rocks, gets the separation numbers, applies the separation numbers, adds the left rock, and then adds the right rock.

Final Actions – still required

At the end of the sim after final output, clean up the shot list. Call deleteSome and give a criteria function that always returns true so that all shots are deleted. Print a diagnostic showing how many shots were deleted.

DIAGNOSTIC: cleared 1 shots from list

The Linked List

Shots are kept on a linked list. Initially, use the supplied link list library and code to it. Late on, write the functions that we use (covered later). Here we go over the basics of the list.

Your code will manage a linked list for shots.

Your sim structure keeps the head pointer. The sim structure uses void pointers for this because the application code does not know the internal data type used by the list. *Even though node.h is in the zip, you may **not** use it in lab 4!* The supplied library implements the list operations for you .

The list provides 3 operations that we use:

- insert (to add an object to a list)

- deleteSome (to delete zero or more objects from a list)
- iterate (to do something with every object, one at a time)

Details on these are in their own sections.

Head Pointer Operations

Your code may do the following to the head pointer that the sim structure holds:

- Set it to NULL (do this once when you initialize the sim structure)
- Compare it to NULL (a NULL value indicates an empty list)
- Pass it to a list function such as iterate that doesn't need to change them (iterate, count, and sort)
- Pass its address to list functions that do need to change them such as insert and deleteSome

Your application code is **not** allowed to manipulate the head pointer in any other way. "Application code" means anything outside the linkedlist.c file. Inside that file, there can be no references to the application. The linked list **can't** include structs.h or similar.

Data on the list

The list will hold any kind of data that you can get the address of. Your code hands a pointer to your data to the list. The list stores that pointer in a void pointer because the list doesn't know what kind of data you want to keep on the list. You can put strings on the list. You can put structs on the list. You should never put different kinds of data on one list at the same time. Your code should always know what data type is on a particular list. In lab 4, that will be pointer to shot structures. In your prototypes, I suggest using strings; they are natively pointer based, they declare without use of malloc, and they are easy to print.

Callback functions

Your code will pass to the list various callback functions. Those callback functions will receive one or more void pointers. Your code **cannot** cast the void pointers, it must assign them to a pointer of the correct type. An example of this is given in the section *Interactions Between Structs and the Lists*.

The callbacks below are described in terms of shots. For prototypes, you can have a different set of callbacks that work with strings. Note that you will *never* need to declare any variable to be of type ActionFunction even though you could. You can pass the name of any function fitting that signature to the list as your action function.

Do **not** declare any variable of the typedef'd types:

- **ActionFunction**
- **ComparisonFunction**
- **CriteriaFunction**

The list is NOT re-entrant safe! For example, your code calls iterate. The iterate function calls your action function "foo." The foo function must never call any list changing functions on the *same* list. Our shots inflight do not interact with each other, so there is no need to re-enter the list. We only have one list.

Action Functions

```
typedef void (* ActionFunction) ( void *data);
```

Action functions are used to do things to the data you stored on the list. In the lab, the action function is given a pointer to a structure. An example action function would be one that draws something. Another use of an action function is to deal with the data held by a node that is about to be deleted. In that case, it should be freed. (In other scenarios it would be OK to insert it onto a different list.)

Comparison Function

```
typedef bool (* ComparisonFunction) (void *data1, void *data2);
```

A common comparison function takes two pointers to object and returns a Boolean indicating true when the first object belongs earlier in the list than the second object. You will write comparison functions and pass their address to parameters of this type. Insert needs these to put new objects into the lists in the proper order. Sort needs them as well.

Criteria Functions

```
typedef bool (* CriteriaFunction) (void *data, void *helper);
```

A criteria function is used to give back a Boolean value. The data value will be a pointer to an object structure. The “always true” criteria function will ignore its parameters. Use “always true” when clearing a list at the end. Other criteria functions will do things like decide if a shot has left the playing field. There could be more complex criteria functions that evaluate the objects with the help of some other data item given in the helper parameter. In this lab, we pass NULL as the helper parameter since none of our criteria need any help; they can look at the object and decide from there.

Library functions

The following functions are provided in the linked list library. See the zip file on piazza. The header file is in the zip as well.

Insert

```
/* int returns FALSE when it fails to do the insert */  
bool insert(void *p2head, void *data, ComparisonFunction goesInFrontOf);
```

Because of call-by-value, the first parameter is the address of the head pointer of the list you want to insert the alien on. The data parameter will be a pointer to an object. When inserting into the inflight list, use a comparison function that uses Y values.

deleteSome

```
int deleteSome(void *p2head, CriteriaFunction mustGo, void *helper,  
               ActionFunction disposal);
```

This function is used to remove nodes from a linked list. It gets passed the address of a head pointer since it may need to alter that pointer. The criteria function passed yields true if the object needs to be removed. The helper pointer will get passed to the criteria function to help it make a decision about each object.

One use of deleteSome is to remove a shot that has left the field. The criteria function decides if the shot needs to leave the list. If so, the action function will be called to dispose of the shot. Other uses of deleteSome are to clear a list at the end.

deleteSome returns the total number of nodes deleted. This number will get printed when your cleanup code runs at the end and clears everything off the linked list.

Iterate

```
void iterate(void *head, ActionFunction doThis);
```

Iterate walks through the list, passing each data item on the list in turn to the action function. The action function does whatever it does to the structure and then iterate goes on to the next node in the list. (In prototypes, the action function does whatever it does to the data type on the list, typically strings.) Since iterate does not cause any nodes to enter or leave the list, the first parameter is a head pointer *not the address of the head pointer like insert and deleteSome need*.

Other Functions

There are other functions in the linked list library, but we won't use them.

Building with the list library

Your makefile might need something like this:

```
LIBS= -llinkedlist -lrx -lncursesw -lm
```

Prototypes and the list

The list really doesn't care what type it holds, as long as your code coughs up a pointer to that data. The easiest way to do prototypes with the list – strongly suggested – is to use strings. They are pointer friendly and print easily and recognizably. Consider this code:

```
void *list = NULL;
rval = insert ( &list, "Go Bucks!", first_letter);
```

Here list is a pointer to void, first_letter is a comparison function that compares the first character of two strings and returns the result of numerically comparing them. Then call

```
iterate(list, print_string);
```

Here print_string is an action function that prints a string (pointer to char) that came in as a pointer to void. It is all of two lines long; a definition of a char pointer and a printf statement.

Don't bother with structures when prototyping the list. Make sure that you can use all of the required list functions. Start with insert and use iterate to test it. Then do a few insertions and delete one thing from the list and use iterate to print what's left. You might insert stirrings that start with both upper and lower case characters and you delete strings that start with a lower case letter. You could test sort by using a comparison function that compares the strlen of the two strings and then using iterate to print the newly sorted list.

Interactions Between Structs and the Lists

The list stores incoming data in void pointers. The list does not know about object structures. When the list wants to give an object back to your code, it will pass the void pointer it holds. **Never cast a void pointer.** As soon as possible, assign that void pointer value to a strongly typed pointer. The list will call your action functions, comparison functions, and criteria functions and they will get passed void pointers.

Your code will have action functions that looks like:

```
void draw_a_shot(void *data)
{
    struct Shot *shot = data;
    /* rest of function goes here.
    * Expect to see shot -> expressions */
}
```

All of your action functions should follow this template. Name your structures as you like, the list will only see void pointers. We know they are pointer to shot structure, so we assign them that way. That lets us use -> to get to the members.

The sim structure and the list

Each void pointer in the sim structure is a list head (we only have one of them). **C is pass by value.** We will never pass the pointer to the sim structure itself to the list. We will be passing a list head to the list, either a copy of it or its address. Inside the list, those pointers are actually pointer to node, but our code doesn't know what a node is because that's not our lab code's business. The list hides nodes from the application. Likewise, the application hides the objects from the list. Neither side knows the internal data type of the other side, this is a truly generic list.

Functions that change the list heads

The **insert** and **deleteSome** functions both need to be able to *change* the head pointer, so we pass the **address** of the head pointer to those functions. The type we pass is pointer to pointer to void. The list uses a simple void pointer to receive that since *a void pointer can hold the value from any pointer type*. Inside the list internals, it will assign that void pointer to a strongly typed pointer to a pointer variable.

The point here is that you have to be able to take a pointer to a structure, use arrow to get a member of the structure, and then & to get the address of that member of that structure. Pass that address.

Functions that don't change the list heads

The **iterate**, **any** and **sort** functions do not add or remove nodes, so the head will not need to change. We pass a *copy* (pass by value is fine here) of the list head to iterate, count and the sort function. Again, the list code receives this in a void pointer and the list knows what strong type to use internally. That happens to be pointer to node, but our lab code doesn't know and shouldn't care.

So your code will take a struct pointer, use the arrow operator to get to a member of the struct and pass it without taking its address.

Clarifications

Color names should be accessed via an array that is indexed by color number as a subscript. Do not use switch-case.

All of your code that touches the linked list should include the libll.h file. Even your own linkedlist.c file should include libll.h as well as linkedlist.h that is generated by the headers.sh utility. This insures that your list code exactly matches the library.

We won't be using sort and you don't have to code it.

Initializations

The init sequence is more complex. See the section on command line arguments. Before the sim is run, the arguments have to be right. Init should check for the presence of arguments but not validate the filename – that’s input’s job. So master init has 2 parts,. Check arguments first before any attempt at calling rx_initialize, there is no reason to kick into graphics mode if the sim isn’t going to run.

Implementing the Linked List (optional 10 point bonus)

This is different than using the linked list, covered above. If sorely pressed for time, delay this functionality and have your code rely on the library.

In lab 4 you will write the 3 functions that we use in the linked list library:

- insert
- iterate
- deleteSome

Signatures

Your code **must** use the exact signatures used by the library functions; same name, same parameters, same return type.

Your code **must** use the following declaration for a node:

```
typedef struct Node{
    struct Node *next;
    void *data;
}Node;
```

The node declaration should **not** be visible outside your linkedlist.c file. This is the same declaration that the library code uses. If you use it, you can mix calls into your code and calls into the library code freely. If you use anything else, you risk going insane trying to track down mysterious pointer bugs in code that looks perfect.

Files

All of your linked list code should be in your linkedlist.c file. All supporting functions must be declared static (internal linkage), leaving only the functions listed above visible outside the file. The headers.sh script should work correctly (it doesn’t create declarations for internal linkage functions).

Dynamic Memory for the list

Allocation and free calls should be inside a function that has a static int to count the number of allocations and frees. So you might want an “allocate_node()” and “free_node()” function with appropriate parameters and return types. They must be in linkedlist.c, they must have internal linkage. Your code should print diagnostics to stderr:

```
DIAGNOSTIC: 1 nodes allocated.
DIAGNOSTIC: 1 nodes freed.
```

Your list code must tolerate allocation failures. If insert fails to allocate a node, return false and let the application deal with it. This *will* happen when you implement the unreliable memory code (See Unreliable).

Visibility

The list code must **not** #include the structs.h or equivalent file. In short, the list code must not know in any way what the data is. The list deals in void pointers and only void pointers for data.

No code outside linkedlist.c is allowed to know what a **node** is.

Only the functions that provide the public interface to the list are allowed to be visible outside the list code. As before, use the static keyword to hide functions.

Plan of Attack

Until you have all the required functions completed, use **-llinkedlist** in your makefile LIBS variable:

```
LIBS= -lrx -lncursesw -lm -llinkedlist
```

This lets you use your code (linkedlist.c) as well as the library (for functions you haven't written yet). Do the easy ones first:

1. iterate
2. insert
3. deleteSome

You don't have to deal with dynamic memory until you get to insert and deleteSome. You could get 4 prototypes done with the list code alone.

Consider having your list functions issue a debug print message so that you know when your code is being used as opposed to when the library is being used.

Once you have the 4 required functions written, **remove** the **-llinkedlist** item in your makefile and test to be sure that your code is being used.

Terse **UPDATED**:

You are allowed to use the terse insert code from the slides, properly modified, for your insert. If you use terse insert, you are **required** to use terse delete. Terse insert is quite short. Terse delete is something like 8 lines. Long-form delete is quite long and somewhat painful to write since the function can delete more than one node.

Update: You are **required** to use terse insert and delete if you implement the list.

Ten-line Limit

If you use the long method for insert, your insert does not have to meet the ten-line limit. If you use the long form of deleteSome, that function does not have to meet the ten-line limit.

Unreliable Allocation

Shots will be stored on the linked list. Shots will be dynamically allocated one at a time and so will the nodes that hold them. The code that allocates shots **must** deal with allocation failure. The code that allocates nodes likewise **must** deal with allocation failure.

We won't test realloc failures. It would be a good idea to do so, but it means that the lab stops right there. Real production code would test the rock code to make sure that it simulated failures coming from realloc. We won't because we will get a flavor of it with nodes and shots and because it would make the lab too complex.

Dealing with allocation failures

In lab 4, your code will be subject to allocation failures. Instead of calling malloc or calloc, it will call modified versions found in yet more libraries. The reliable library works as expected. The unreliable library fails at "random" times, causing objects and nodes to fail to allocate. Your code must gracefully deal with these events as described here.

You will need to download altmem.zip.

Recovery

Your code should not quit upon memory failure. If a shot fails to allocate, return and carry on. If a node fails to allocate, insert returns false and the lab 4 code will have to free the allocated object since the list doesn't have a node to hold it. This is *a* recovery strategy and not *the only* recovery strategy. It presumes that our code took measures to make future allocations work and it presumes that rolling over and playing dead at the first sign of difficulty is not acceptable. The Apollo 11 Eagle spacecraft went into computer overload on descent to the moon but the software was built to keep the important stuff alive even if it meant the other stuff starved. One of the questions to ask when you are given a programming assignment is, "what is our error recovery strategy?"

Details

Code Changes

If a shot fails to get inserted onto the linked list, it must be freed.

Change every malloc, calloc, and free call to use the "alternative" equivalents instead.

Note that the "libraries" are not actual libraries. They are .o files that have been compiled with -g so that you can run gdb without issues. Here is the altmem.h header file found in the altmem.zip archive in piazza:

```
/* alternative memory calls (both reliable and unreliable use these) */
void *alternative_calloc(size_t nmemb, size_t size);
void *alternative_malloc(size_t size);
void alternative_free(void *ptr );
```

You will want to

```
#include <stdlib.h>
#include "altmem.h"
```

in the 2 files you have that deal in dynamic memory. Any file that includes altmem.h must *first* include stdlib.h. Note also that other header files might require stdio.h. When including files, put standard headers <stdio.h> before custom headers “altmem.h” [note the <> versus “”].

There will be point deductions if any of the non-prototype code you turn in directly calls malloc, calloc. Any free calls for nodes or shots must also be converted. (Does not apply to realloc and the call to free for the rocks array).

Your shot dynamic memory code should be restricted to one file on the application side. Likewise, your linkedlist.c file should be the only file that deals in dynamic memory for nodes.

What follows is example makefile lines as guidance for lab4. It builds the lab4 target – a mandatory target – using the reliable memory allocation code. The lab4u target builds against the unreliable memory allocation code. Note the **highlighted** items. Modify the dependency list to suit, but leave either reliable or unreliable in there (but not both).

a lab 4 makefile prior to writing linked list code might resemble this

```
lab4u: lab4.o n2.o (other .o files here) unreliable.o
      gcc -g -o $@ $^ $(LD_FLAGS) $(LIBS)
```

```
lab4: lab4.o n2.o (other .o files here) reliable.o
      gcc -g -o $@ $^ $(LD_FLAGS) $(LIBS)
```

Prototypes for altmem

It might be worth doing some prototypes that call malloc or calloc repeatedly to test out how this stuff works and to see some code that detects NULL go into action.

Initial testing for allocation failures

The unreliable allocation code has the following sequence:

Fail. Succeed one time. Fail. Succeed two times. Fail. Succeed three times...

For us this means:

Using the linked list library where nodes always allocate	Using your own linked list code where nodes might fail to allocate
The first shot <i>fails</i> to allocate	The first shot <i>fails</i> to allocate
The second shot allocates and gets put on the list	The second shot allocates
The third shot <i>fails</i> to allocate	The node for the second shot <i>fails</i> to allocate (insert returns false)
The fourth shot allocates and gets put on the list	The third shot allocates
The fifth shot allocates and gets put on the list	The node for the third shot allocates and it goes on the list
The fifth shot <i>fails</i> to allocate	The fourth shot <i>fails</i> to allocate

Reference code issues

It's quite possible that the reference code doesn't deal with allocation failures even if it detects them. Be very sure that your code does properly deal with these issues. Check the DIAGNOSTIC numbers at

the end to make sure. Check every insert call in the reference code as well as the ball and block allocation sequence.

Bonus #1: Shots hitting rocks: 10 points

In order to keep the amount of required code under control, the code for shots hitting rocks and rocks undergoing fission is a ten-point bonus. (Firing and moving shots and taking them off the board is still required)

Have a symbol in debug.h ROCK_BONUS and set it to 1 or 0 to control what happens. There should be a single function call in your function that handles motion related activities. In my code that function is “all_shots_hit_rocks” and it is the only place where you need to worry about the ROCK_BONUS symbol.

This mode is controlled by recompiling.

Bonus #2: Bonus Mode: A Playable Game: 10 points

Bonus 2 requires bonus 1. It is activated on the command line.

Most of the changes for bonus mode can be restricted to a bonus.c file but there will be hooks in the rest of the code. The sim needs a bool so that various other places can tell if bonus mode is enabled.

- Game on needs to ignore the time limit when in bonus mode. The game runs until all of the rocks are gone or all of the ships are gone.
- Ship actions will call for the bonus actions instead of the regular actions when in bonus mode.

The bonus actions function will start a chain of activities. It should loop reading characters via rx_getch(). If rx_getch returns a character, it should ask for it to be processed. If rx_getch returns ERR (-1), the loop is over and the function returns.

Each character to be processed should be used to consult a keymap. The keymap has 256 entries but it is sparsely populated, with only a 12 entries filled in. If the key maps to a filled in entry, the entry will indicate an action to take and the team it applies to. Send the action index and team color to the function the will act.

The act function has a 4-entry jump table of functions that take a ship and return nothing. Use the team color number to locate the correct ship. If that ship has a non-zero color, it is still in the game (it hasn't been crunched) and pass that ship pointer to the function in the jump table that has the correct action index. (We have 4 actions in the jump table).

Here is the **keymap**:

Action index	Team 1	Team 2	Team 3
0	a	j	4
1	s	k	5
2	d	l	8
3	f	;	6

Here is the **jump table**:

Turn left
Fire
Boost
Turn right

So if the user presses the 'a' key, the ship for team one, if it is still in the game, will take action zero, which is turn left. You will need to write the two turning functions and the boost function. You should already have a function in ships.c that is given a ship and fires a shot.

Your code might have lines like the following:

```
map['a'] = (struct Keymap){1, LEFT};
```

You might want both tables to be static. The jump table can be initialized at the point of declaration, but the keymap can't easily be initialized that way. Keeping a static bool allows you to call your keymap initialization code just once. The code that loads the keymap is exempt from the two-line limit; you have 12 assignments to make.

The left turning function subtracts 15 degrees from the ship angle. The right turning function adds 15 degrees to the ship angle. In both cases, keep the result in the range of $0.0 \leq \text{angle} < 360$.

The boost function adds 1 to the ship's overall velocity vector in the direction specified by the ship's angle. In code terms, add cosine of the angle to the ship's Y velocity and sin of the angle to the ship's X velocity. Convert degrees to radians as the argument to sin or cos. Those are declared in math.h. Also print a boost message showing the X and Y velocities. (The supplied output.c has the boost message function).

Playing the game takes some concentration. The angle is displayed as a number next to the score for each ship. The leading crosshairs are an approximation of how the ship is pointing, but the angle is exact. Beware of boosting to high speeds (this was also a problem in the original). There is no limit to shots, so hammering one of the turn keys alternating with the fire key can get you out of danger. Boost happens in the direction the ship is pointed at, not the direction it is traveling.

[Bonus #3, 10 points – implement the linked list, see above](#)

Test Files

When running in graphics mode, be sure to redirect file descriptor 2 as in:

```
./lab4 xZB.rx xZB.mytext 2>xZB.mydnd
```

The file xZ1.rx is good for basic shot and fission testing. It is also good for testing unreliable allocation. The shot comes in horizontally, hits straight on with the shot direction pointed at the rock center, and the 2 new rocks separate vertically. Unreliable allocation will mean that it takes successive tries to get the first shot off.

The file `xZ0.rx` is good for testing shots in regular (no bonus at all) and first bonus modes. It is short – time limit is 1 second. In both modes the ships shoot twice and one of them wraps position. It also has a rock-rock bump early on.

The file `xZ5` has a horizontal line of rocks that get hit a glancing blow by a shot that is traveling horizontally but is slightly higher than the rock centers. The rocks split into two, each part going diagonally. The rocks that go upwards do not get clear of the shot before it hits them as well. It has a longish run time, test with `xZ1` first.

The file `xZ8.rx` has a line of rocks that are too small to fission when they are shot, so a single shot destroys them all.

The file `xZB.rx` has a fleet of three ships making small rocks out of big rocks. It's fun to watch in graphics mode. It makes sure that your code can handle lots of stuff at the same time; 3 ships means 3 shots at a time and a growing and shrinking collection of rocks. In the end, the cyan rock manages to crunch all three ships after 29 seconds of runtime. You can also play this in three-player bonus mode.

The file `xZS.rx` has a red ship (team 1) and one huge rock. Great for single player bonus mode. Note that when bigger rocks split, their children have higher separation velocities than the children of smaller rocks. Use the asdf controls. Post your high score and a screenshot to Piazza.

No Global Variables! Code Must Compile! No Magic Numbers!

Global variables is a -50 penalty.

Errors or warnings in compilation means no credit for the lab, though any credit for prototypes you demo will remain.

“Magic” numbers should be `#define` symbols with descriptive names. Some constants are **not** magic:

- The 0.5 in `kinetic = 0.5 * mass * velocity * velocity;`
- Most uses of 0 and 0.0 are not magic

The above are not deemed magic because:

1. They never change, regardless of units.
2. They are easy to type and verify at a glance.
3. Their meaning is always clear by context and never changes

Magic numbers include:

- The value of pi. Use `M_PI` (found in `math.h`) – pi fails test #2 above.
- 4.0 seconds used as the delay time for final graphical output – it fails #1 and #3.
- Various dimensions and values used in the simulation such as delta time.

Submission

Effectively: Same as before. Your zip file needs to contain README_LAB4, **all** the code to be graded, and a makefile sufficient to build all of the targets that are to be graded. **Those targets include lab4 and lab4u.** Do not include any of your .o files. **Do** include supplied .o files such as n2.o, reliable.o, unreliable.o. Any file you edited by hand (other than test data) probably needs to be included. Also include the libraries (*.a) in your zip.

Your lab is counted late if the makefile doesn't self-test the zip or if the self-test has warnings about missing files if those files were supplied to you (n2.o, librx.a...). You get a zero if the missing file is one of yours.

All files that you edit by hand must have your name in them. Machine generated headers need your name in them.

Any code that comes from the instructor's lab 2 or 3 must retain proper attribution.

Defects in the submission can cause a zero score for the lab or worse.

README_LAB4 text:

THIS IS THE README FILE FOR LAB 4

BY SUBMITTING THIS FILE TO CARMEN, I CERTIFY THAT I HAVE PERFORMED ALL OF THE WORK TO DETERMINE THE ANSWERS FOUND WITHIN THIS FILE MYSELF WITH NO ASSISTANCE FROM ANY PERSON OTHER THAN THE INSTRUCTOR OF THIS COURSE OR ONE OF OUR UNDERGRADUATE GRADERS. NONE OF THIS CODE WAS GENERATED BY AI.

The readme should contain your **name**, the number of **hours** you worked on the lab, and any comments you want to add. Of particular interest are what was hard or easy about the lab or places where programming reinforced what we went over in class.